CIND 119 Final Project: Credit Risk Classification

Toronto Metropolitan University

Problem Statement

Credit risk assessment is a crucial task in the banking and financial sector. Poor credit decisions can lead to significant financial losses, increased default rates, and reduced profitability. On the other hand, overly strict lending policies may result in missed business opportunities. Therefore, developing an accurate and reliable predictive model for credit risk classification is essential.The ability to segment customers and predict the customer's credit worthiness, through data analytics, machine learning & predictive modeling methods, would be a great business advantage for our client.

Introduction

In this project, we take on the role of data scientists working to solve a real-world financial problem for a banking institution. The objective is to analyze customer financial data and predict credit risk specifically, whether a customer is classified as high risk or low risk, and consequently, whether a loan should be approved or rejected.

Thus, this project aims to find a solution to our client's "customer creditworthiness" business problem by Exploratory Data Analysis (EDA) and Predictive Modeling/Classification techniques (Classification using Decision Tree, Naïve Bayes). The dataset we are using is "german_credit.arff". There are 1000 rows, 21 columns/attributes in this dataset.

Introduction

In the first part of the project, we will carry out Data Preparation to understand the data and identify the research questions such as: “How is the data distributed?”, “Which attributes seem to be correlated?”, “Which attribute can be included/eliminated in the analysis based on statistics?” and so on. Within the second part, we will build the classification machine learning models based on the Decision Tree and Naïve Bayes algorithm.

The main tools used in the scope of this project are Python, its packages for Data Analysis, Visualization, Machine Learning (Pandas, Numpy, Matplotlib, Seaborn, Pandas Profiling, Scikit-learn, Imbalancedlearn) along with additional visualization tools

Introduction

This project is divided into four (4) main phases:

1. **Date Preparation**: Clean up any missing data or incorrect data types
2. **Exploratory Data Analysis (EDA)**: Understand the structure of the dataset, identify key variables, and examine relationships between features using visuals. Ex. Which variables influence credit risk the most? and “What patterns exist among high-risk customers?”
3. **Predictive Modeling**: Apply supervised machine learning classification techniques, specifically **Decision Tree** and **Naïve Bayes** algorithms, to build predictive models that classify customers based on their creditworthiness.
4. **Conclusions and Recomendations**: Evaluate the machine learning models, share insights, and make recommendations to the bank.

Solution/Methodology

To address the credit risk prediction problem, a structured data science approach was followed:

1. Data Preparation & Exploration

- Loaded and inspected the German Credit dataset.
- Checked for missing values, data types, and inconsistencies.
- Performed Exploratory Data Analysis (EDA) to understand distributions and relationships.
- Visualized key variables such as credit amount, duration, and customer status.
- Identified important features influencing credit risk using correlation analysis and statistical insights.

2. Data Preprocessing

- Encoded categorical variables into numerical format for model compatibility.
- Normalized and scaled features where necessary.
- Split the dataset into training and testing sets.
- Addressed class imbalance using appropriate techniques (SMOTE).

Solution/Methodology

3. Model Building

a) Decision Tree Classifier

- Built a Decision Tree model to classify customers into high-risk or low-risk categories.
- The model provides clear interpretability by showing decision rules.
- Evaluated performance using metrics such as accuracy, precision, recall, and confusion matrix.

b) Naïve Bayes Classifier

- Implemented a Naïve Bayes model based on probability theory.
- Assumes independence between features, making it computationally efficient.
- Particularly effective for classification tasks with categorical data.

Solution/Methodology

4. Model Evaluation

- Compared both models based on:
  - Accuracy
  - Precision & Recall
  - Confusion Matrix
- Identified which model performs better in predicting high-risk customers.

5. Business Interpretation

- Translated model results into business insights:
  - Identified key risk factors (e.g., high loan amount, longer duration of credit).
  - Suggested criteria for loan approval decisions.
  - Highlighted how predictive modeling can reduce financial risk.

Workload Distribution

Member Name	Tasks Performed
Richard, Brent (brent.richard@torontomu.ca)	Data Prep & EDA, Organizing & Completing the Project Report
Shaban, Abdullah (a1shaban@torontomu.ca)	Naïve Bayes Models, Organizing & Completing the Project Report
Rani Debi, Rima (rima.ranidebi@torontomu.ca)	Decision Tree Models, Organizing & Completing the Project Report

Data Preparation

Data Preparation

German Credit Card Dataset	
Rows	1,000
Attributes (count)	21
Data Types	Quantitative (6), Qualitative (15)
Missing values	None
Count of Nulls	0
Duplicate Rows	None

Data Preparation

Data type	Data type Description	Count of Attributes	Attributes
Qualitative	Nominal	14	• Creditability • Account Balance • Payment Status of Previous Credit (Credit History) • Purpose • Value Savings/Stocks • Length of current employment • Sex & Marital Status • Guarantors • Most valuable available asset • Concurrent Credits (instalment plans) • Type of apartment • Occupation • Telephone • Foreign Worker
	Ordinal	1	• Duration in current Address
Quantitative	Continuous	1	• Credit Amount
	Discrete	5	• Age (years) • Duration of Credit (month) • Instalment % • No. of Credits at this bank • No. of dependents

Data Preparation

Created a cleaned up dataframe for use with Naive Bayes and Decison Tree

Data Type conversion

Converted **‘Instalment per cent’** to **datatype: object**

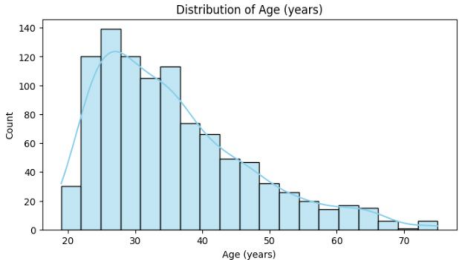
Log Normalization

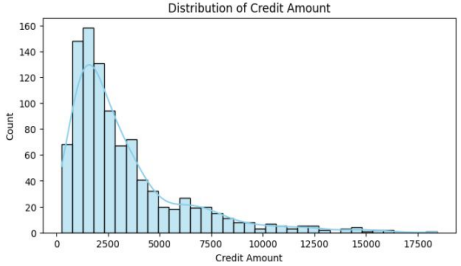
Naïve Bayes works best when numerical attributes are normally distributed. We applied a Log normalization to three categories:

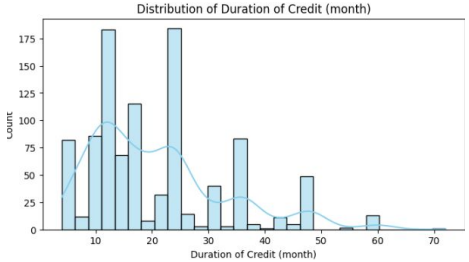
- Age (years)
- Credit Amount
- Duration of Credit

Data Prep- Log Normalizing

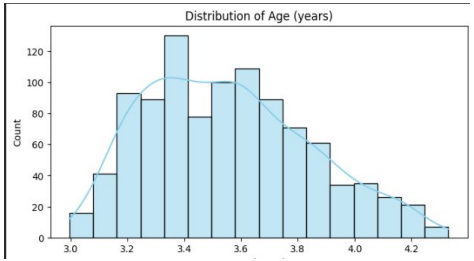
Original Distribution

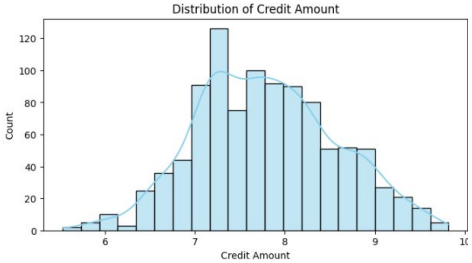


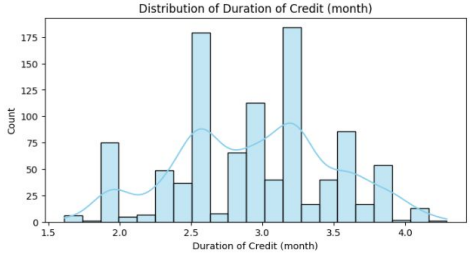




Log Normalization

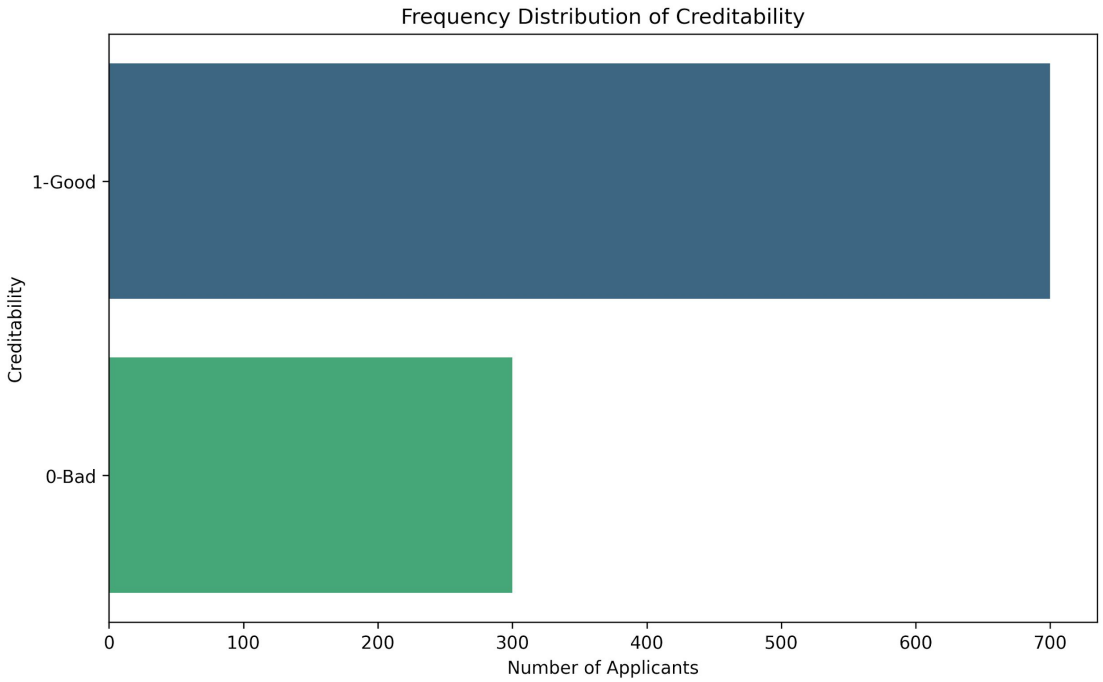




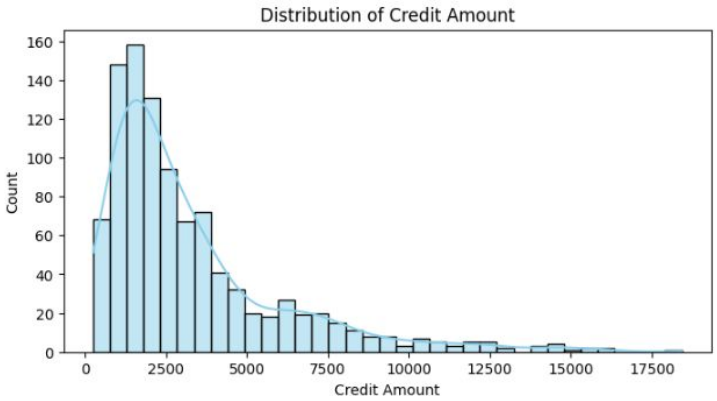


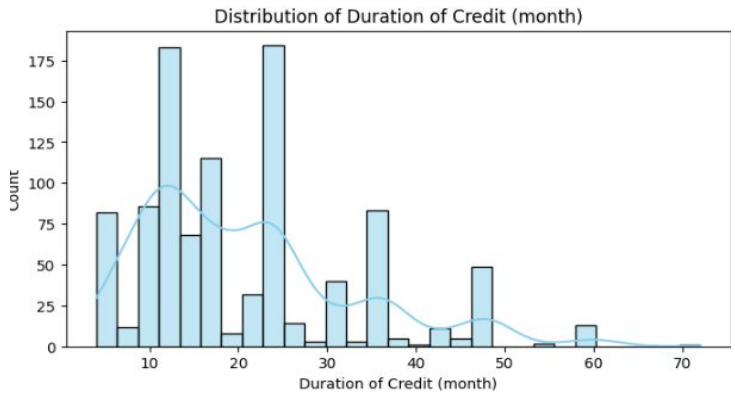
Exploratory Data Analysis

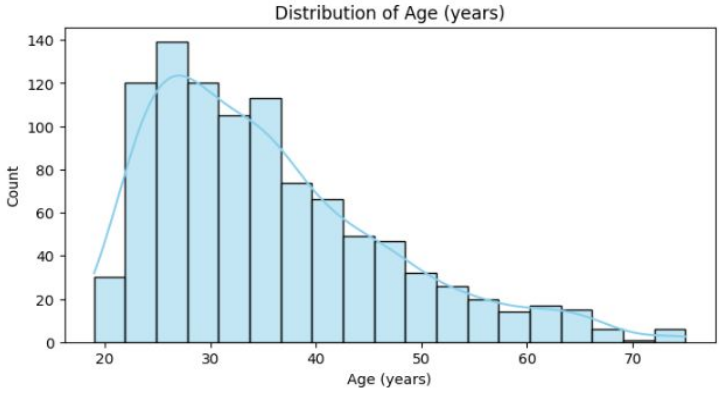
Exploratory Data Analysis- Distributions



Exploratory Data Analysis- Numeric Distributions

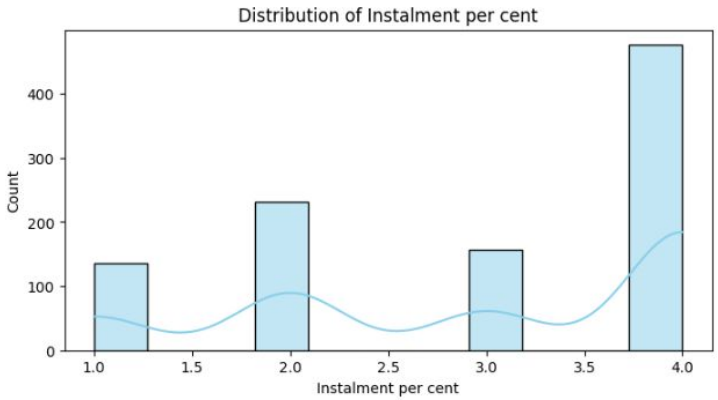


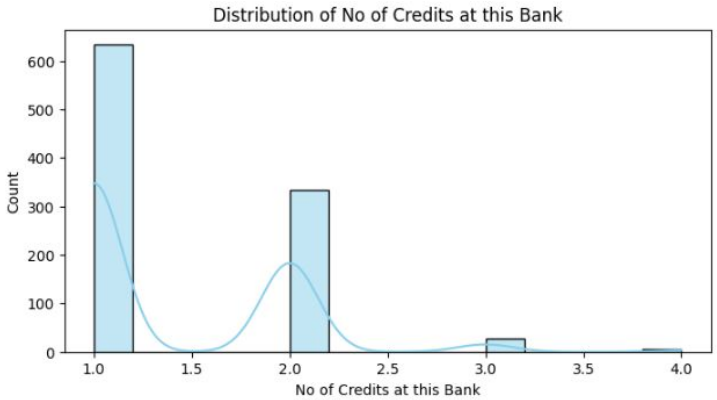


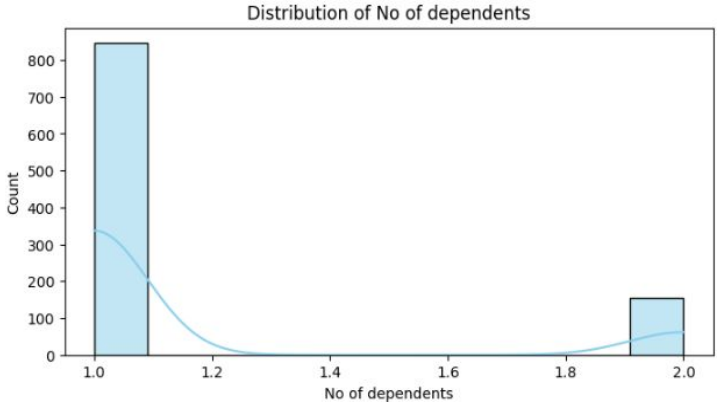


	Mean	Median	Mode	Min	Max	STDev
Credit Amount	3,271.25	2,319.5	1,258.0	250	18,424.0	2,822.75
Duration of Credit (month)	20.9	18	24	4	72	12.05
Age (years)	35.5	33	27	19	75	11.35

Exploratory Data Analysis- Numeric Distributions

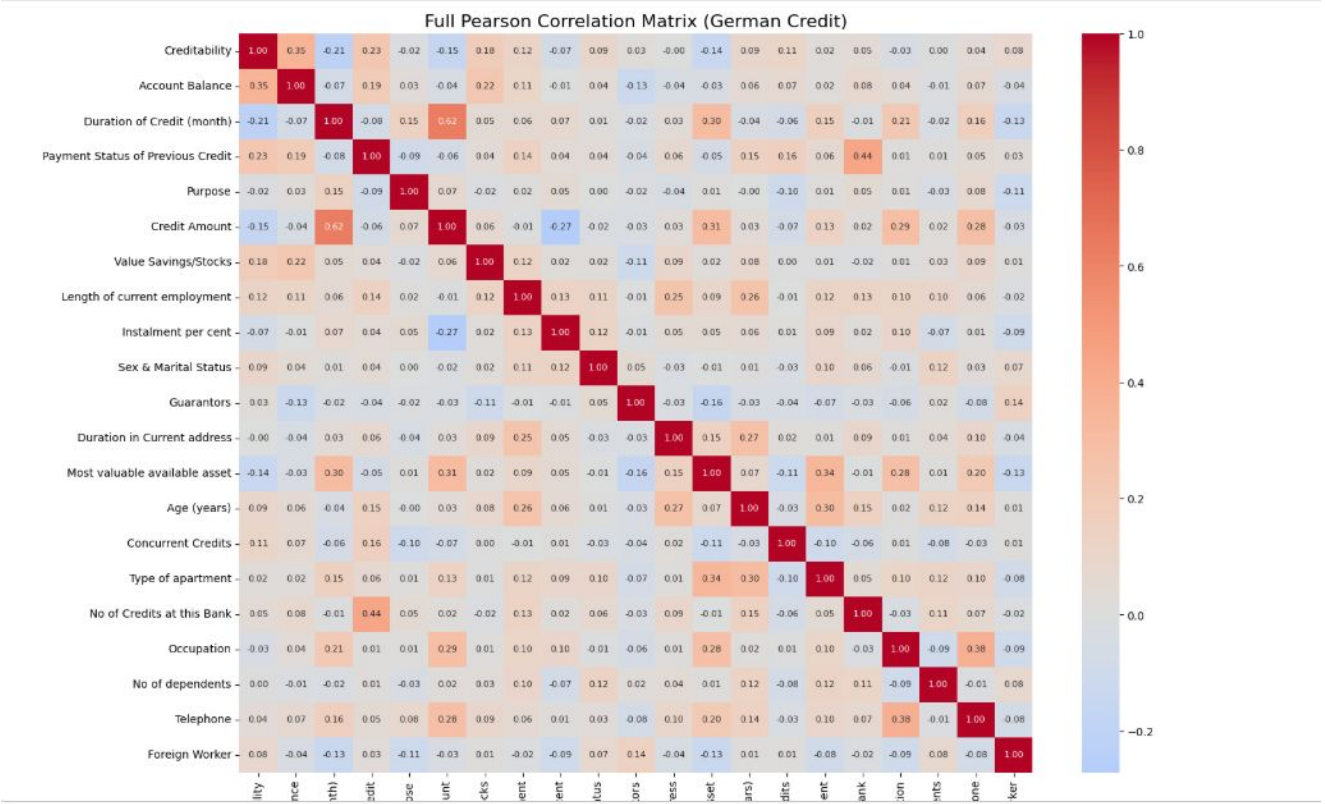




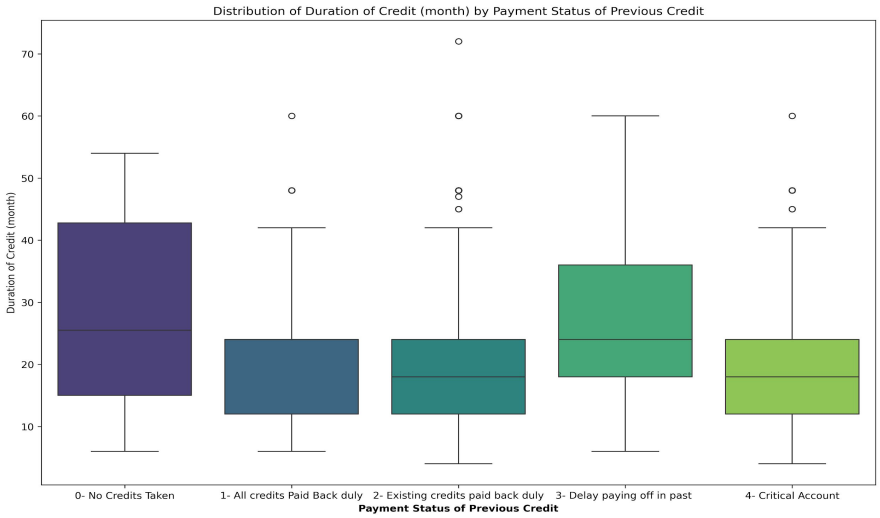


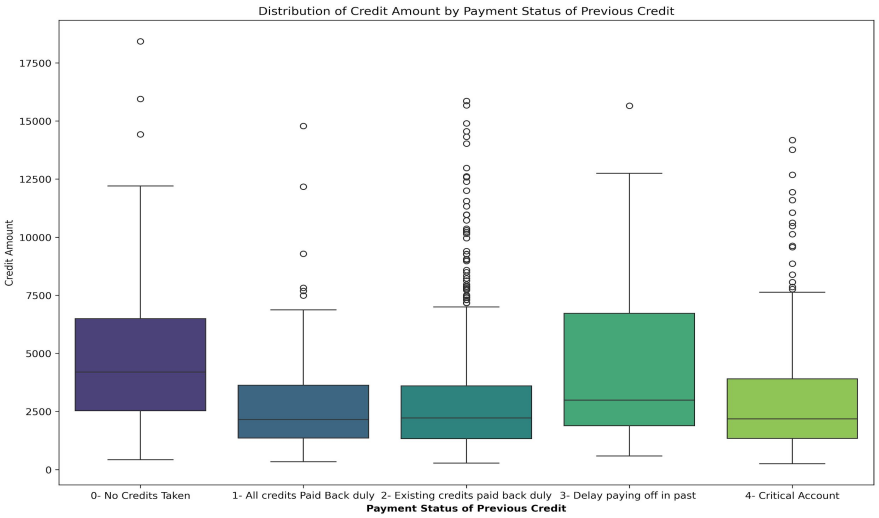
	Mean	Median	Mode	Min	Max	STDev
Instalment per cent	2.97	3	4	1	4	1.11
No of Credits at this bank	1.4	1	1	1	4	.57
No of Dependents	1.15	1	1	1	2	.36

Correlation Heatmap



Exploratory Data Analysis- Distributions





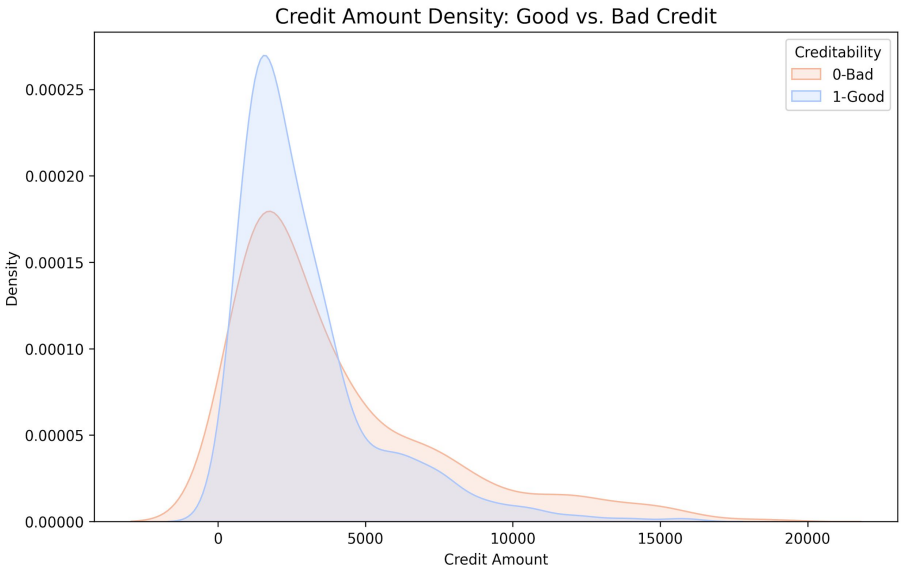
Exploratory Data Analysis- Correlation Good Credit

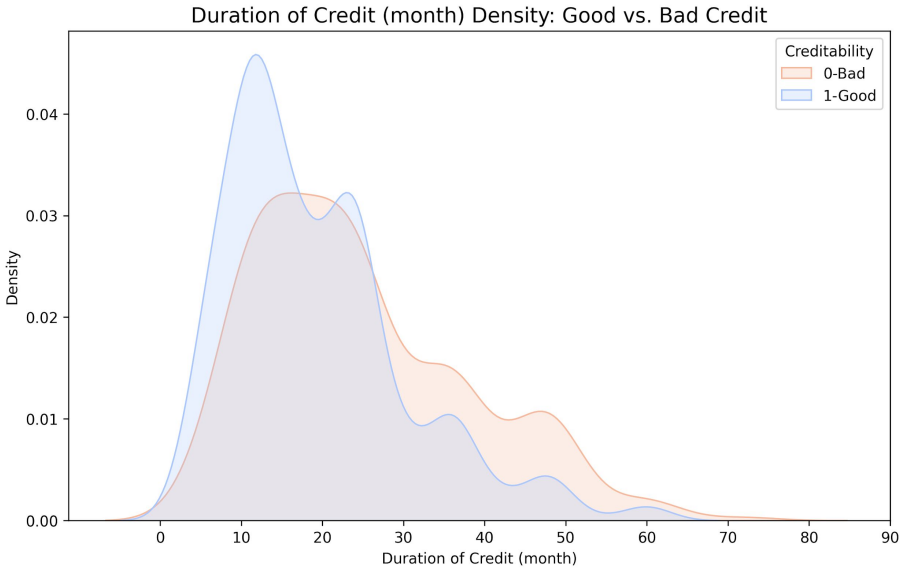
Attribute	
Creditability	1.000
Account Balance	0.351
Payment Status of Previous Credit	0.229
Value Savings/Stocks	0.179
Length of current employment	0.116
Concurrent Credits	0.110
Age (years)	0.091
Sex & Marital Status	0.088
Foreign Worker	0.082
No of Credits at this Bank	0.046
Telephone	0.037
Guarantors	0.025
Type of apartment	0.018
No of dependents	0.003
Duration in Current address	-0.003
Purpose	-0.018
Occupation	-0.033
Instalment per cent	-0.072
Most valuable available asset	-0.143
Credit Amount	-0.155
Duration of Credit (month)	-0.215

- Account Balance shows the **strongest positive correlation** to Good Credit. As Account Balance code increases, the likelihood of Good Credit increases (code 4: No Checking Account does not necessarily mean "No wealth"

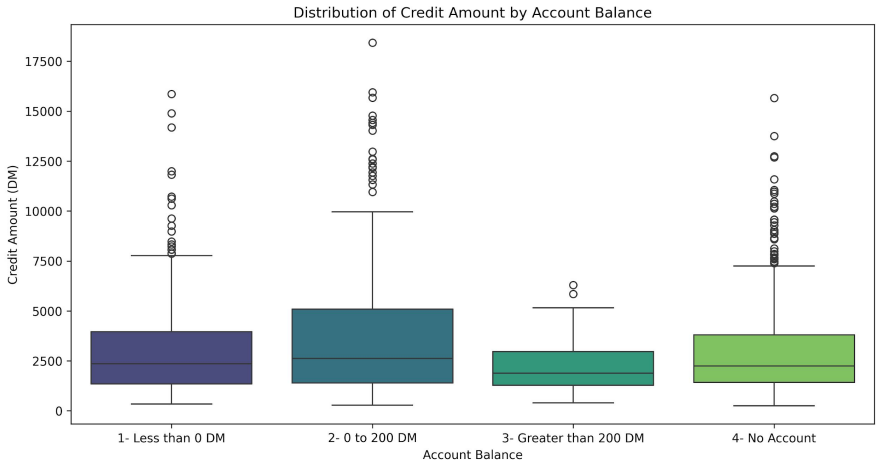
- Credit Amount and Duration of Credit show the strongest negative correlations to Good Credit

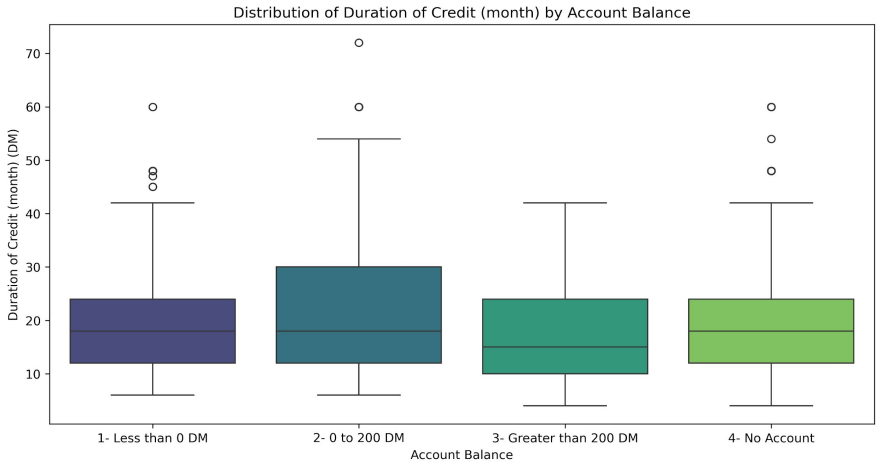
Exploratory Data Analysis- Kernel Density Plot





Exploratory Data Analysis- Distributions





Predictive Modeling

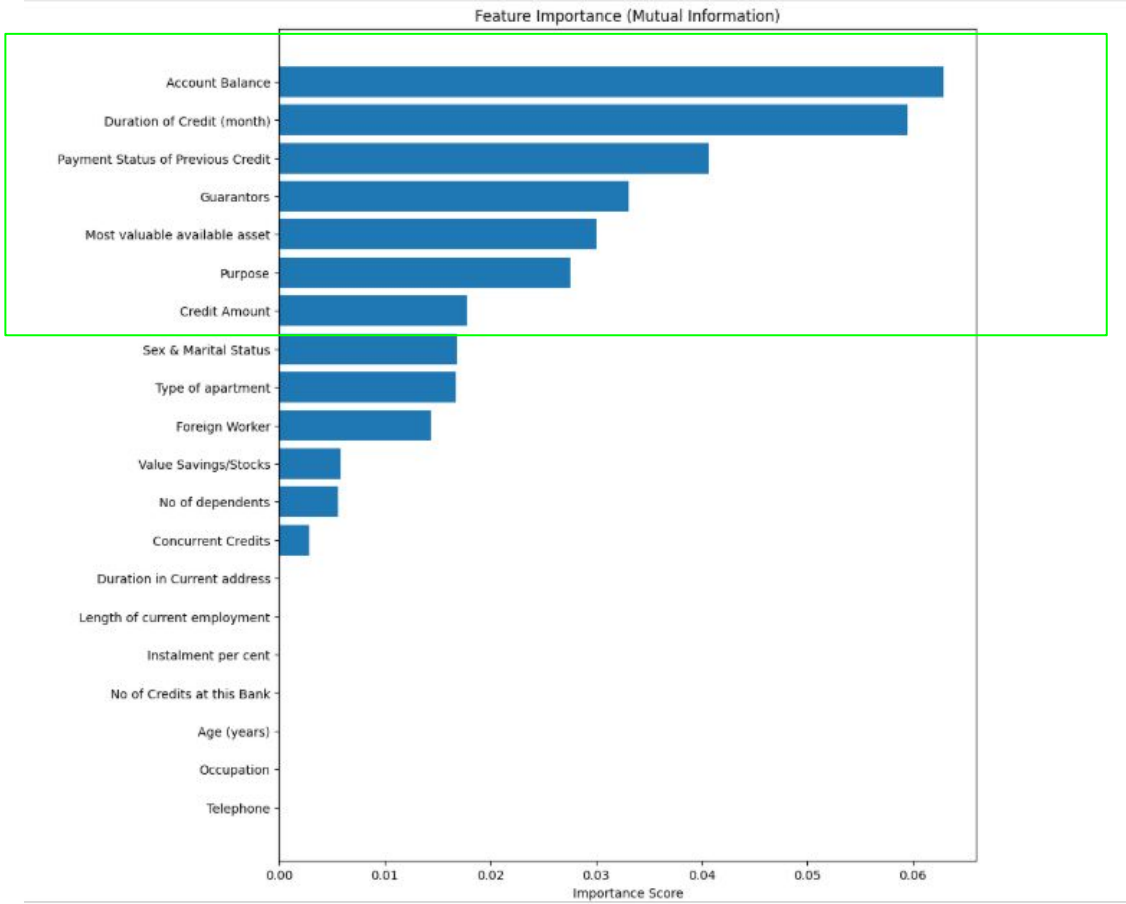
Data Splitting Strategy

Splitting the dataset is a critical step before building any machine learning model, as it ensures that the model is properly trained and evaluated on unseen data.

80/20 split	• resulted in slightly higher accuracy due to increased training data • Use 80/20 when maximizing model learning is important
70/30 Split	• provided a more reliable and stable evaluation due to a larger test set • Use 70/30 when a more reliable evaluation is needed

We chose to move forward with the 80/20 split, because our data set size was limited. We want to maximize the accuracy of our models.

Feature Selection



Feature Selection

In this project, feature selection for the Naive Bayes model was performed using Python's built-in feature_importance_ functionality from a tree-based model (such as a Random Forest classifier).
The feature_importances_ function works by measuring how much each feature contributes to reducing impurity (e.g., Gini impurity or entropy) across all the decision trees in the model. Every time a feature is used to split the data, the algorithm calculates how much that split improves the classification.

These improvements are accumulated over all trees and normalized, resulting in a score for each feature. Features that consistently lead to better splits and improved predictions receive higher importance values. Based on these scores, features are ranked from most to least important, and only the top-performing ones are selected for training the Naive Bayes model. This ensures that the model focuses on the most relevant variables, reducing noise and improving efficiency.

Predictive Modelling (Naive Bayes)

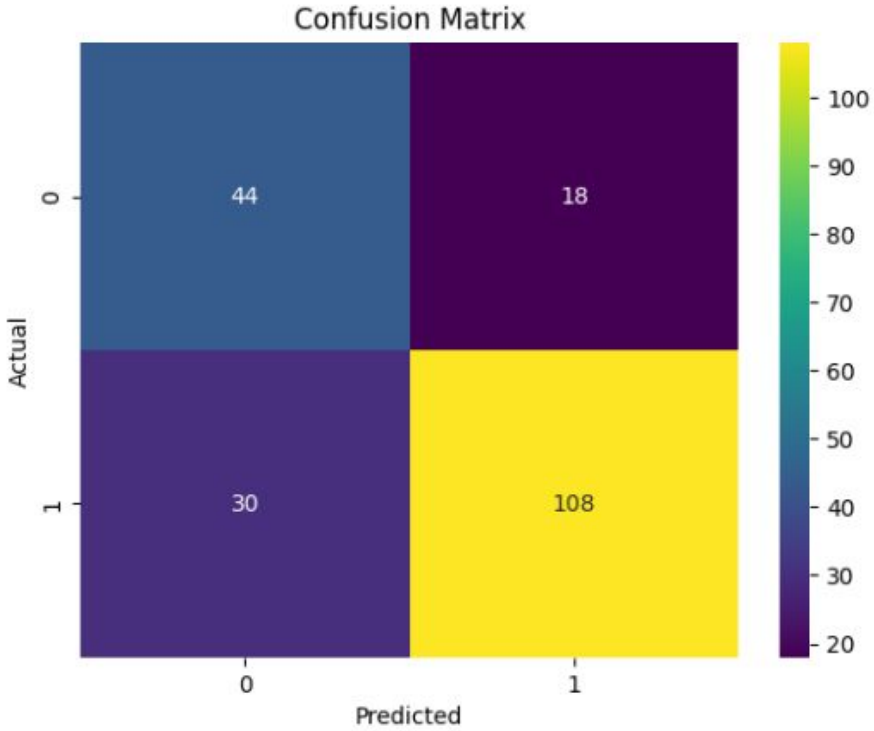
- Each row in the dataset represents a customer, and each column represents an attribute
- The model assumes feature independence and predicts whether a customer is high risk or low risk based on the highest probability.
- This problem falls under binary classification, where there are two possible outcomes.
- For the German Credit dataset, the two class labels represent:
  - **Low Risk (Good Credit)**: indicating that the loan is likely to be approved
  - **High Risk (Bad Credit)**: indicating that the loan is likely to be rejected

Naive Bayes baseline model with all features evaluation metrics

The Accuray score of Baseline Naive Bayes Model with all features included and a split of 80/20 is: **0.765**

	precision	recall	f1-score	support
0	0.59	0.71	0.65	62
1	0.86	0.78	0.82	138
accuracy			0.76	200
macro avg	0.73	0.75	0.73	200
weighted avg	0.78	0.76	0.77	200

Naive Bayes baseline model with all features evaluation metrics- 80/20

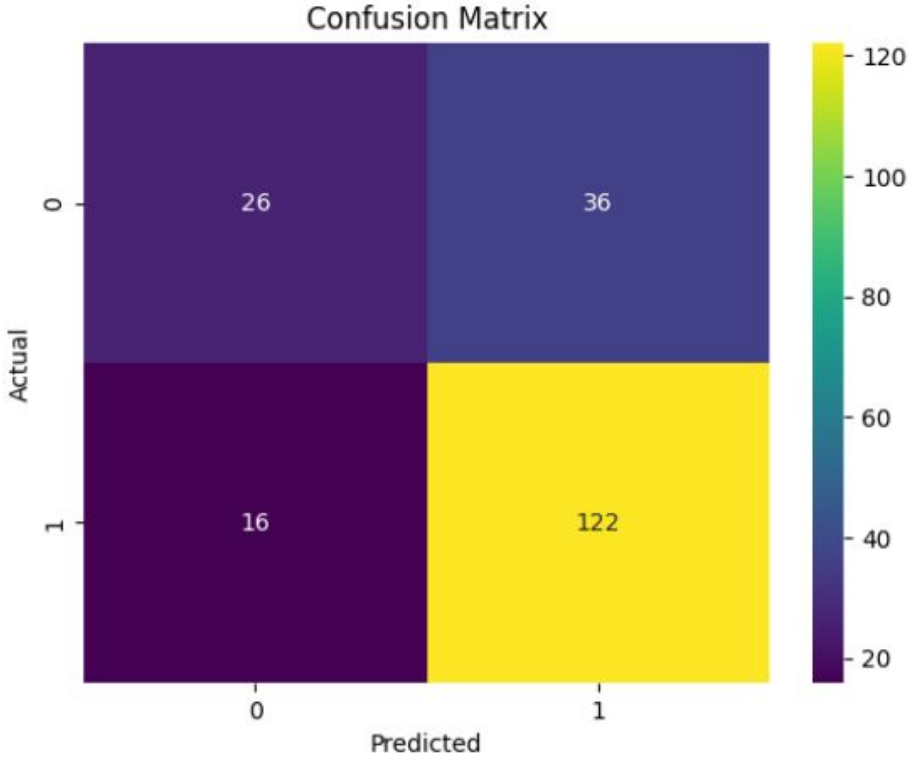


Naive Bayes model with selected features evaluation metrics

The Accuracy score of Naive Bayes Model with selected features and a split of 80/20 is: **0.74**

	precision	recall	f1-score	support
0	0.62	0.42	0.50	62
1	0.77	0.88	0.82	138
accuracy			0.74	200
macro avg	0.70	0.65	0.66	200
weighted avg	0.72	0.74	0.72	200

Naive Bayes model with selected features evaluation metrics- 80/20

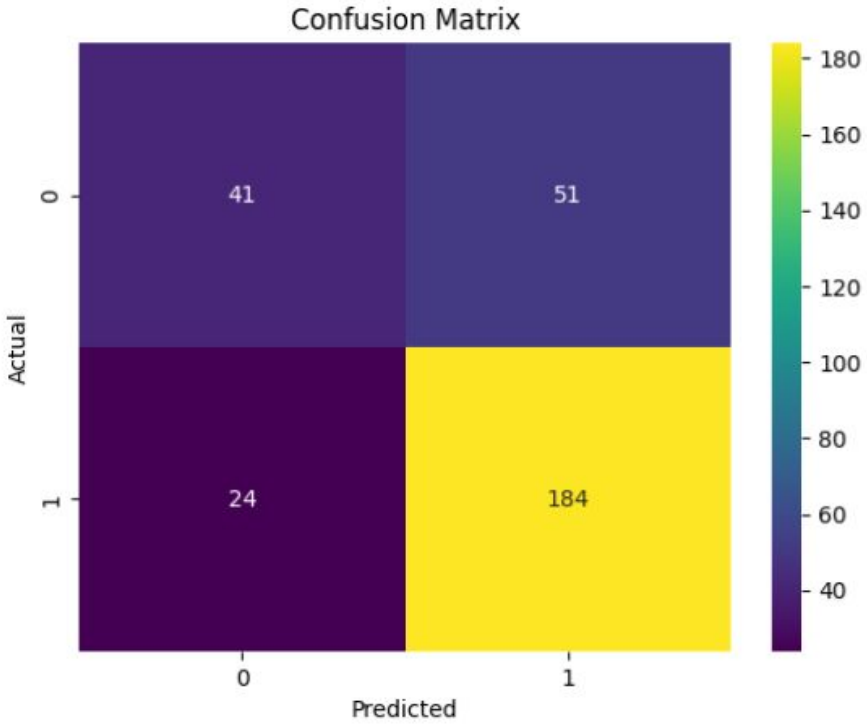


Naive Bayes model with selected features evaluation metrics

The Accuracy score of Naive Bayes Model with selected features and a split of 70/30 is: **0.75**

	precision	recall	f1-score	support
0	0.63	0.45	0.52	92
1	0.78	0.88	0.83	208
accuracy			0.75	300
macro avg	0.71	0.67	0.68	300
weighted avg	0.74	0.75	0.74	300

Naive Bayes model with selected features evaluation metrics-70/30

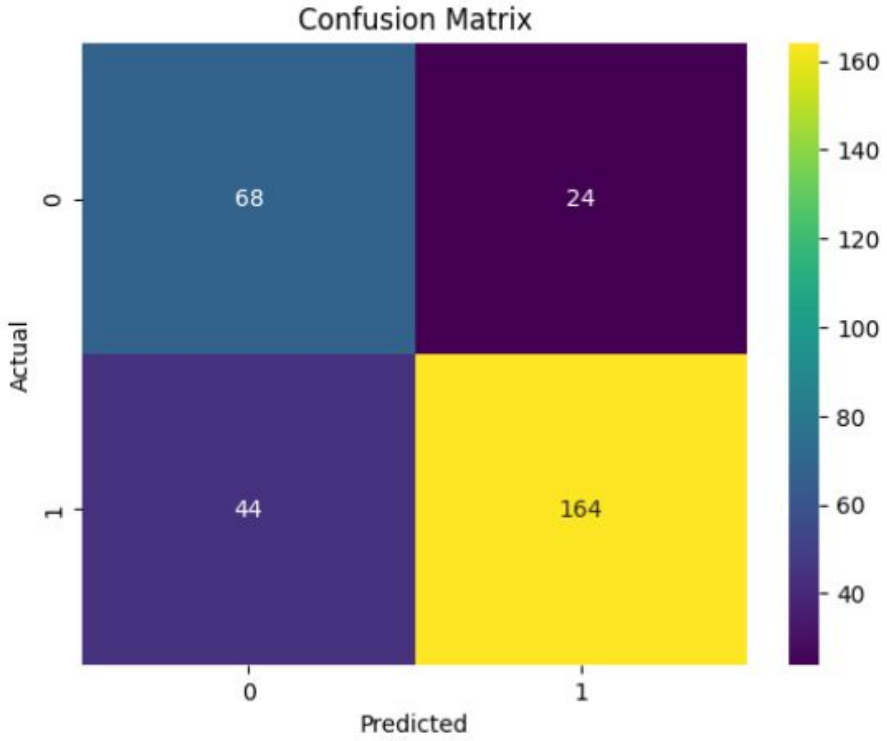


Naive Bayes baseline model with all features evaluation metrics

The Accuracy score of Baseline Naive Bayes Model with all features included and a split of 70/30 is:
0.7733

	precision	recall	f1-score	support
0	0.61	0.74	0.67	92
1	0.87	0.79	0.83	208
accuracy			0.77	300
macro avg	0.74	0.76	0.75	300
weighted avg	0.79	0.77	0.78	300

Naive Bayes baseline model with all features evaluation metrics- 70/30



Class Imbalance Problem (Solved Using SMOTE)

The Accuracy of Naive Bayes Model with SMOTE and a split of 80/20 is: **0.63**

Classification Report:

	precision	recall	f1-score	support
0	0.44	0.83	0.57	60
1	0.88	0.54	0.67	140
accuracy			0.63	200
macro avg	0.66	0.69	0.62	200
weighted avg	0.75	0.63	0.64	200

Class Imbalance Problem (Solved Using SMOTE)

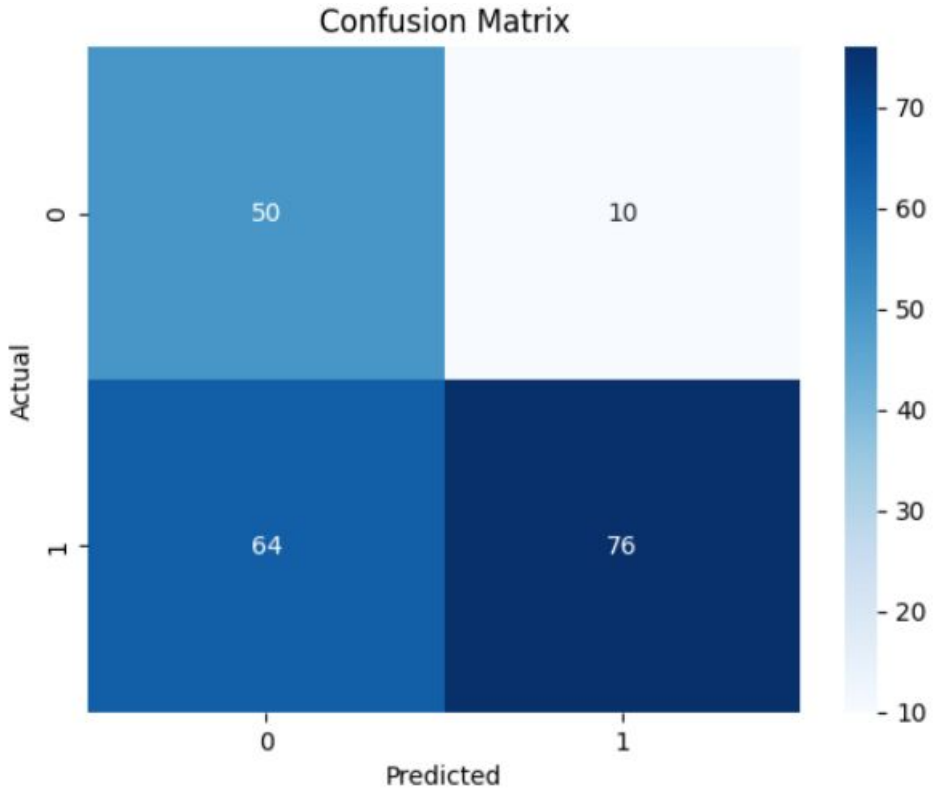
Dataset before SMOTE:

```
Before SMOTE:
Creditability
1    700
0    300
Name: count, dtype: int64
```

Dataset after SMOTE:

```
After SMOTE:
Creditability
1    560
0    560
Name: count, dtype: int64
```

Class Imbalance Problem (Solved Using SMOTE)



Class Imbalance Problem (Solved Using SMOTE)

In the German credit card dataset, a clear class imbalance was present, with 700 records labeled as good credit(1) and 300 records labeled as bad credit (0). Such imbalance can significantly affect the performance of our machine learning models, as algorithms tend to favor the majority class and may incorrectly predict most cases as good credit, reducing the model's ability to accurately identify risky applicants. This bias can lead to poor detection of bad credit customers, which is a critical aspect of credit risk assessment. To address this issue, the Synthetic Minority Oversampling Technique (SMOTE) was applied. SMOTE works by generating synthetic examples of the minority class rather than simply duplicating existing records. By applying SMOTE, the dataset was balanced to 560 good credit records and 560 bad credit records, allowing the model to learn patterns from both classes more effectively and improving its ability to make fair and reliable credit predictions

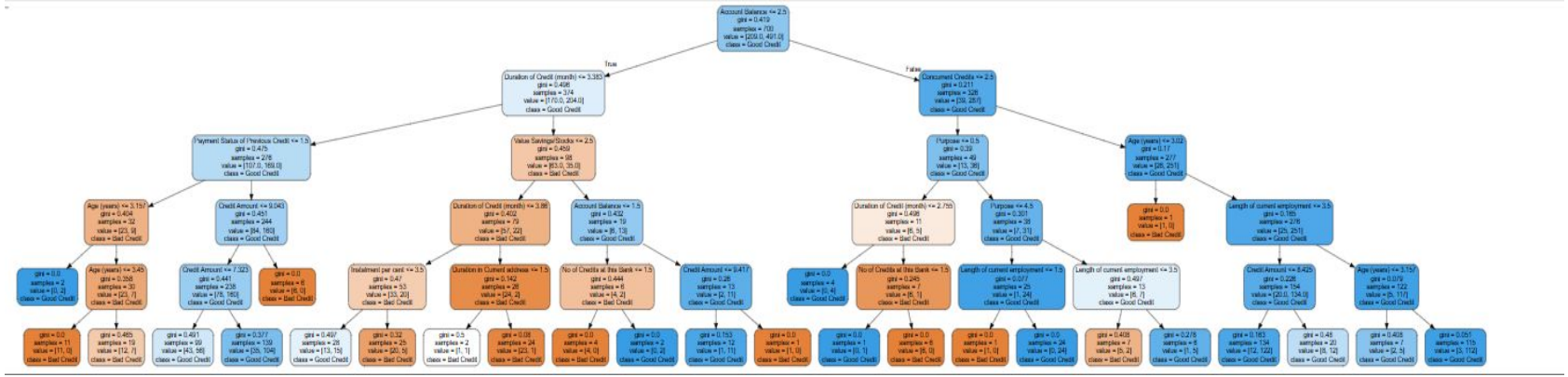
Class Imbalance Problem (Solved Using SMOTE)

While SMOTE successfully balanced the dataset and improved the model's ability to detect the minority class (bad credit), it also led to an increase in false negatives. This happens because SMOTE generates synthetic samples of the minority class, which can sometimes introduce overlapping patterns between the classes.

As a result, the model may become slightly more uncertain when distinguishing between good and bad credit cases, leading to more instances where actual bad credit applicants are incorrectly classified as good credit (false negatives). Although the increase in false negatives indicates a trade-off, the use of SMOTE still helps the model learn more balanced decision boundaries and reduces the strong bias toward the majority class.

Classification using Decision Tree

We developed a classification decision tree model to predict customer's credibility (binary outcomes: 1– for Good Credit and 0– for Bad Credit) based on the dataset. The output diagram of the decision tree will provide more detail about the nodes and splits in a capped five-level tree. The default grow algorithm here is gini. When our team run the decision tree with all features, balanced class labels on "Credibility" using the training set (max depth of 5) in Python; below is a snapshot of the baseline decision tree model.



The decision Tree Baseline Model (80/20 Split)

The Accuracy of Baseline Classification Tree Model with a 80/20 split is: **0.70**

0	0.46	0.30	0.37	56
1	0.76	0.86	0.81	144
accuracy			0.70	200
macro avg	0.61	0.58	0.59	200
weighted avg	0.68	0.70	0.68	200

```
Confusion Matrix
[[ 17  39]
 [ 20 124]]
TP:  124 , FP:  39 , TN:  17 , FN: 20
```

The decision Tree Selected Features Model (80/20 Split)

The Accuracy of Classification Tree Selected Features Model with a 80/20 split is: **0.70**

```
Confusion Matrix
[[ 17  39]
 [ 21 123]]
TP:  123 , FP:  39 , TN:  17 , FN: 21
              precision    recall  f1-score   support

           0       0.45      0.30      0.36        56
           1       0.76      0.85      0.80       144

    accuracy                           0.70       200
   macro avg       0.60      0.58      0.58       200
weighted avg       0.67      0.70      0.68       200
```

The decision Tree Baseline Model (70/30 Split)

From the test dataset, we also have to check if the baseline decision tree model works well in predicting the customer's credibility, using accuracy, precision and recall as evaluation metrics. The accuracy score, the classification report along with the recall and f1 score can show us whether the model did a great job at predicting the customer's creditworthiness.

The Accuracy of Baseline Classification Tree Model with a 70/30 split is: **0.76**

	precision	recall	f1-score	support
0	0.76	0.32	0.45	91
1	0.76	0.96	0.85	209
accuracy			0.76	300
macro avg	0.76	0.64	0.65	300
weighted avg	0.76	0.76	0.73	300

Confusion Matrix

```
[[50 10]
 [64 76]]
```

The decision Tree Selected Features Model (70/30 Split)

The Accuracy of Classification Tree Selected Features Model with a 70/30 split is: **0.76**

```
Confusion Matrix
[[ 29  62]
 [  9 200]]
TP:  200 , FP:  62 , TN:  29 , FN: 9
              precision    recall  f1-score   support

           0       0.76      0.32      0.45        91
           1       0.76      0.96      0.85       209

    accuracy                           0.76       300
   macro avg       0.76      0.64      0.65       300
weighted avg       0.76      0.76      0.73       300
```

Predictive Modeling- 80/20 Split

	Naive Bayes Model		Decision Tree	
	All Features	*Selected features*	*All features*	*Selected Features*
Accuracy	76.5%	74%	70%	70%
Precision	78%	72%	68%	70%
Recall	76%	74%	67%	70%

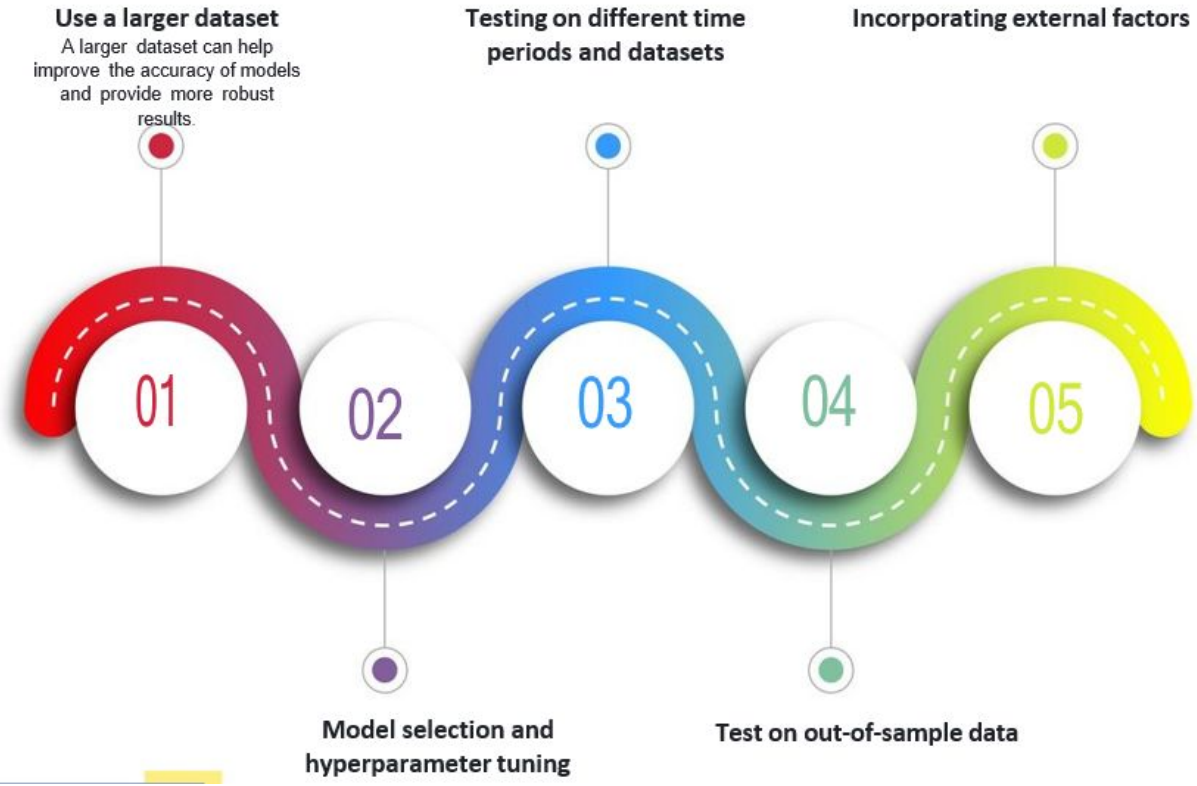
Conclusion and Recommendation

- By far, the best predictive result for customer creditworthiness characterization and prediction comes from utilizing all the attributes in the dataset to build a Naive Bayes classification model with an **accuracy score of 76.5%** and a **precision rate of 78%.**
- The decreasing performances going from using all features down to several selected features in Naive Bayes reflect a need to further investigate and select a better combination of selected features. This implies that choosing the suitable features is very important if we would like to enhance the outcome of this predictive analytics project. Carrying out iterative cycles of data pre-processing and feature selection process may indeed improve our models' performances.
- Another separate step of "Feature Evaluation" can be considered in which we rank which features play the most important role in the classification.

Conclusion and Recommendation

- We can also try applying RandomForestClassifier's "feature_importance" to rank the most important features for a given classification. Moreover, we also recommend exploring other predictive techniques such as K-Means in case they may yield more promising results for customer credit prediction.
- It is also important to note that The Decision Tree's identical performance (70% accuracy) across both models suggests that the selected features likely contained the same primary predictors the baseline model was already using for its top-level splits. Since trees are inherently selective, they naturally ignore noisy or redundant variables. Since the most influential attributes such as credit history or savings were present in both versions, the model generated the same logic and outcomes.
- This indicates that the removed features were likely insignificant to the tree's decision-making process, causing the model to remain structurally unchanged despite a smaller dataset.

Future Work


Use a larger dataset
A larger dataset can help improve the accuracy of models and provide more robust results.
Testing on different time periods and datasets
Incorporating external factors
01
02
03
04
05
Model selection and hyperparameter tuning
Test on out-of-sample data

“Let it be the beginning of the new Journey.”